HighFieldBoat VCB commissioning checklist

Plan: `docs/VCB_COMMISSIONING.md` · commands: `docs/VCB_COMMISSIONING_CHEATSHEET.md` (also the appendix here) · firmware `firmware/VCB-pixhawk` 0.3.1 · printed 2026-10-20

Firmware version flashed ———— Flash dump of the old build ————

Boat location / bench ———— Pack voltage at start ————

Standing rules. ☐ Props OFF through stage 5. ☐ One person at the E-stop, nobody near the motors; the VCB arms and closes the contactors by itself when both inputs sit at zero for 1 s. ☐ The 20 % thrust limit stays until the control system is tested and tuned under autonomy (gate 6). ☐ Abort rule: anything unexpected (a contactor closes when it should not, a motor moves when it should not, `bus_mv` disagrees with the meter, a fault you did not provoke) → E-stop, power-cycle the VCB, read the log before going on. ☐ Do not start a stage before the previous gate is signed.

Per-session header (fill in on each page you use)

Date	Operator	E-stop person	Witness	Weather / notes

Telemetry fields used (from `~/vcb.json` and the raw console)

`status`: `thruster_state` (1 IDLE, 2–4 start-up, 5 RUN, 0 FAULT), `armed`, `lsc_closed`/`hsc_closed`, `bus_mv`, `fault_code`, `estop_24v_present`/`tsms_24v_present`, `cmd_source` (0 PWM, 1 bench). `pwm`: `chN_us`, `chN_valid`, `chN_raw_permille`, `chN_applied_permille`, `chN_bad`, `chN_cmd` (signed byte last sent to the converter; ch1 = right motor, ch2 = left). Fault codes: 1 command lost · 2–7 contactor/bus interlock · 8 contactor opened in RUN (latched) · 9 console `stop` (latched until `clear`).

What you need on the bench

☐ Jetson: `vcb-logger` running, `~/vcb.json` updating, dashboard :8080 ☐ Three terminals: raw console tail, `watch cat ~/vcb.json`, command shell with `vcb()` ☐ Pixhawk on USB, mavlink-router :5760, QGC or MAVProxy (stage 4 on) ☐ Multimeter: HV bus at the contactor output; contactor coil feed ☐ BMS reader (`battery.json`) ☐ Optional: USB-RS485 sniffer on UART5 (38400 8N1) ☐ Optional: servo tester for CONT (3.3 V logic only!) ☐ Tools on the Jetson: `vcb_bench_drive.py`, `vcb_bench_check.py`, `ap_apply.py`

Stage 0 · Firmware and dead-bus checks

Pack disconnected. Can be done now. Commands: cheat sheet §0.

Date ———— Operator ———— Old-build dump file ————

	#	Do	Expect	P / F	Values / notes
☐	0.1	Fresh flash dump; `cmp` against the August dump	Dump saved next to the August one; byte diff count recorded (0 = same build runs today)	☐ P ☐ F	diff bytes:
☐	0.2	Flash 0.3.1	Console banner `VCB pixhawk-pwm reversible 0.3.1`. If silent: try other bauds (UART4/115200 was inferred)	☐ P ☐ F	
☐	0.3	`vcb help`	Command list comes back (console input works)	☐ P ☐ F	
☐	0.4	Apply `params/vcb-pixhawk.parm` (only after 0.2)	SERVO9/10 = 1380 / 1500 / 1620 verified by `ap_apply.py`	☐ P ☐ F	
☐	0.5	Safety switch off / `BRD_SAFETY_DEFLT=0`, reboot Pixhawk	PWM present on AUX1/AUX2 when disarmed (1500 μs)	☐ P ☐ F	
☐	0.6a	`vcb_bench_check.py --pwm 1500,1550,1600,1620,1700,1450,1400,1380,1300,1000,2000`	raw 0, 52, 157, 200, 368, –52, –157, –200, –368, –1000, 1000; applied clamps ±200; LEFT → ch2, RIGHT → ch1; 1000 and 2000 `valid=1`	☐ P ☐ F	
☐	0.6b	`--pwm 990,1000,1010,1990,2000,2010`	raw –1000, –1000, –978, 978, 1000, 1000; all valid; `bad` not counting	☐ P ☐ F	
☐	0.6c	`--pwm 2250,2300,2350,650 --hold 3` (if ArduPilot accepts)	2250/2300 valid, raw 1000; 2350/650 `bad` counts, `valid` 0 after 100 ms	☐ P ☐ F ☐ n/a	

	#	Do	Expect	P / F	Values / notes
☐	0.6d	Pixhawk armed in Manual, stick nudged while VCB armed-but-IDLE; then unplug CONT 1 s, replug	`DISARMED: throttle left neutral before RUN`; then `FAULT 1` → recovery	☐ P ☐ F	
☐	0.7	Timebase: `ch1_us`/`ch2_us` at the 1500 step	1485 .. 1515 μs (else revisit `PWM_DEADBAND_US` before stage 4)	☐ P ☐ F	ch1: ch2:

☐ **Gate 0 passed**: every item above, console input proven, measured neutral inside the deadband.

Date _____ Signed _____ Witness _____

Stage 1 · Safety: every kill path, contactors open

Pack connected, VCB powered, **Pixhawk unplugged from CONT** so the VCB cannot arm (`thruster_state` 1, `armed` 0 throughout). If it arms, stop and find out why. Cheat sheet §1.

Date _____ Operator _____ E-stop person _____

☐ Pixhawk unplugged from CONT ☐ Meter on the contactor coil feed ☐ Props off ☐ Console windows open

	#	Do	Expect	P / F	Values / notes
☐	1.1	E-stop pressed / released × 3	`estop_24v_present` follows; coil feed loses 24 V when pressed	☐ P ☐ F	sense reads ___ pressed / ___ released
☐	1.2	TSMS pulled / inserted × 3	`tsms_24v_present` follows; coil feed loses 24 V when pulled	☐ P ☐ F	sense reads ___ pulled / ___ in
☐	1.3	Main battery disconnect / BMS off	HV dead at the contactor input (meter); BMS reader agrees	☐ P ☐ F	
☐	1.4	E-stop **pressed**; plug Pixhawk in at neutral so the VCB arms and tries to start	`ARMED`, `STARTUP_LSC`, `FAULT 3` within 1 s, `lsc_closed` stays 0; 3 retries, `FAULT latched`. Unplug, reset VCB	☐ P ☐ F	
☐	1.5	Same with TSMS pulled	Same	☐ P ☐ F	
☐	1.6	`vcb stop`, then `vcb clear`	`FAULT 9`, `FAULT latched: stop command`; clear un-latches	☐ P ☐ F	
☐	1.7	Walk the kill chain: what loses power on E-stop?	Written down: converter / motors / coils only	☐ done	

☐ **Gate 1 passed**: both hardware kills remove coil power and block a start-up; software stop works; sense polarity recorded.

Date _____ Signed _____ Witness _____

Stage 2 · VCB arm, start-up, disarm, zero throttle

Pack connected. Power-up order: Pixhawk first (disarmed output 1500 μs arms the VCB), TSMS in and E-stop released, then VCB power. The VCB arms 1 s after boot and starts at once if the bus reads < 5 V. E-stop person ready at VCB power-on. Cheat sheet §2.

Date _____ Operator _____ E-stop person _____ Pack V (meter) _____

☐ Props off ☐ Pixhawk disarmed, outputs 1500 ☐ Bus < 5 V before VCB power ☐ TSMS in, E-stop released

	#	Do	Expect	P / F	Values / notes
☐	2.1	VCB power on, hands off	`ARMED`; state 1→2→3→4→5 in ~2 s; LSC then HSC closed; `bus_mv` rises; `cmd` 0/0	☐ P ☐ F	time to RUN:
☐	2.2	`bus_mv` vs meter at contactor output	Within 1 V (else fix the divider scalar in `config.h` before stage 3)	☐ P ☐ F	bus_mv: meter:
☐	2.3	10 min in RUN, zero throttle	`fault_code` 0; coils and precharge resistor not warm	☐ P ☐ F	
☐	2.4	**E-stop in RUN**	`FAULT` 8 in ~50 ms; both contactors open; latched; stay open after release	☐ P ☐ F	
☐	2.5	Reset VCB, 2.1 again, TSMS pulled in RUN	As 2.4	☐ P ☐ F	
☐	2.6	Reset VCB, 2.1 again, pull CONT in RUN; replug at neutral	`FAULT` 1 within 100 ms, contactors open; then `FAULT cleared` (3 s), `ARMED` (1 s), RUN: no power cycle	☐ P ☐ F	
☐	2.7	Reboot the Pixhawk in RUN	`FAULT` 1; recovery as 2.6 when outputs return at 1500	☐ P ☐ F	
☐	2.8	Pixhawk armed in Manual; nudge throttle during the start-up	`DISARMED: throttle left neutral before RUN`; start-up aborted	☐ P ☐ F	
☐	2.9	`vcb stop` in RUN; then `vcb clear`	`FAULT` 9, contactors open; after clear + 3 s neutral: recovery to RUN	☐ P ☐ F	
☐	2.10	Reset VCB, pack disconnected	`FAULT` 6, 3 retries, `FAULT latched`; precharge resistor cool after 4 attempts	☐ P ☐ F	
☐	2.11	Pixhawk unplugged; `vcb_bench_drive.py --right 0 --left 0 --seconds 5`; again with Ctrl-C mid-way	`cmd_source` 1, `ARMED`, RUN with `cmd` 0/0, `DISARMED: bench off`, IDLE; nothing turns; Ctrl-C exits the same way	☐ P ☐ F	
☐	2.12	Three clean power cycles, pack on	RUN each time	☐ P ☐ F	1 ☐ 2 ☐ 3 ☐

☐ **Gate 2 passed**: RUN 3/3, bus reading trusted, E-stop and TSMS each open the contactors with fault 8, PWM loss and Pixhawk reboot stop and recover, bench-mode arm/disarm clean.

Date _____ Signed _____ Witness _____

Stage 3 · Jetson drives the motors directly, 5 %, props off

Command path: Jetson → `bench R L` on `/dev/vcb`; no Pixhawk in the loop (bench mode ignores the PWM inputs). 50 permille = 5 % = converter byte ±6. Firmware caps bench commands at 10 %. Each row is one `vcb_bench_drive.py` invocation (cheat sheet §3); it aborts to zero on Ctrl-C, fault or stale data.

Date _____ Operator _____ E-stop person _____

☐ **Props OFF, verified by a second person** ☐ Nobody within reach of the motors ☐ Gate 2 signed ☐ BMS reader open

	#	Command	Expect	P / F	Values / notes
☐	3.1	`--right 50 --left 0 --seconds 5`	`ch1_cmd` 6; the **right** motor turns, **forward** (would push water aft). Wrong motor or wrong way: fix in `eprop.cpp`, re-flash, repeat	☐ P ☐ F	motor that turned: R / L direction: fwd / rev
☐	3.2	`--right 0 --left 50 --seconds 5`	`ch2_cmd` 6; left motor forward	☐ P ☐ F	motor: R / L dir: fwd / rev
☐	3.3	`--right -50 --left 0`, then `--right 0 --left -50`	`cmd` –6; each motor reverses	☐ P ☐ F	
☐	3.4	`--right 50 --left 50 --seconds 10`	Both run; no fault; BMS current noted	☐ P ☐ F	pack A:
☐	3.5	`--right 50 --left 0 --seconds 30`	Steady 30 s; bad not moving; converter does not drop the motor	☐ P ☐ F	
☐	3.6	`--right 100 --left 100 --seconds 5`; then `--right 300 --left 0`; then by hand `bench 300 0`	`cmd` 13/13 runs; tool refuses 300; firmware prints `BENCH: clamped to 100 0`	☐ P ☐ F	
☐	3.7	30 s run; `kill -9` the tool from another shell	≤ 500 ms later `FAULT 1`, `DISARMED: bench command lost`, motor stops, contactors open	☐ P ☐ F	
☐	3.8	**E-stop during a run**	`FAULT 8`, motor stops or coasts. Record what actually stopped it (contactors cut motor power, or only the stop frame)	☐ P ☐ F	stopped by:
☐	3.9	`vcb stop` during a run, then `vcb clear`	`FAULT 9`, motor stops	☐ P ☐ F	
☐	3.10	`--right 50 --left 0 --seconds 3 --flip`; same on the left	`cmd` 6 → 0 (200 ms) → –6; motor reverses, no dropout	☐ P ☐ F	

☐ **Gate 3 passed**: motor identity and direction known (firmware corrected if needed), both directions on both motors, both together, 30 s steady, every stop path with a motor turning stops it.

Date _____ Signed _____ Witness _____

Stage 4 · Pixhawk motor test, props off

Pixhawk on CONT, `vcb bench off`. Each row: `vcb_bench_check.py --live --side right|left --pwm <µs> --hold <s>` (requires VCB in RUN, no fault; typed confirmation). Start every step from RUN with `cmd` 0/0. Cheat sheet §4.

Date _____ Operator _____ E-stop person _____

☐ **Props OFF, verified** ☐ Gate 3 signed ☐ Pre-arm checks pass (motor test arms the Pixhawk)

	#	Command	Expect	P / F	Values / notes
☐	4.1	right, 1550 µs, 2 s	`ch1_raw` ≈ 52, `ch1_cmd` 7, the right motor forward. If left turns: swap SERVO9/10_FUNCTION (73↔74)	☐ P ☐ F	
☐	4.2	left 1550; right 1450; left 1450	`cmd` 7 / –7 / –7; left forward, each reverses	☐ P ☐ F	
☐	4.3	right, 1620 µs, 10 s	`ch1_cmd` 25, steady, no fault	☐ P ☐ F	pack A:
☐	4.4	**End points**: right 2000 µs 30 s; right 1000 µs 30 s; same on the left	raw ±1000, applied ±200, `cmd` ±25, `bad` not counting, motor runs the whole 30 s	☐ P ☐ F	R+ ☐ R– ☐ L+ ☐ L– ☐
☐	4.5	2200, 2300, 800, 700 µs (servo tester if ArduPilot refuses); then 2350 / 650	First four valid, raw ±1000; 2350/650: `bad` counts, `valid` 0, `FAULT 1`	☐ P ☐ F ☐ n/a	
☐	4.6	right 1620 then 1380 back to back; same on the left	`cmd` 25 → 0 (200 ms) → –25; reversal at 20 % without a dropout	☐ P ☐ F	
☐	4.7	Optional: `DIRECTION_CHANGE_STOP_MS` 0, re-flash, repeat 4.6, put it back	Tells whether the hold is needed; if the converter drops the motor without it, keep the default	☐ P ☐ F ☐ n/a	hold needed: Y / N
☐	4.8	Scale check at 1620 µs	Display % / power / BMS current consistent with 20 % of scale. If it looks like 25 %: full scale is 100 → `EPROP_FULL_SCALE_VALUE` 100	☐ P ☐ F	full scale: 127 / 100 W:
☐	4.9	ArduPilot disarm while the VCB is in RUN	Outputs 1500; VCB stays RUN with `cmd` 0, contactors closed (by design). **Decide** whether acceptable	☐ seen	decision:

☐ **Gate 4 passed**: PWM mapping confirmed, 30 s at each end point without a dropout on each side, reversal at 20 % without a dropout, full scale known, 4.9 decided.

Date _____ Signed _____ Witness _____

Stage 5 · RC commands, props off

Needs the RC receiver installed (ELRS XR3) and calibrated; Manual mode; `params/vcb-pixhawk.parm` applied; `MOT_SLEWRATE` 100 to start. Cheat sheet §5.

Date _____ Operator / pilot _____ E-stop person _____

☐ **Props OFF, verified** ☐ Gate 4 signed ☐ Receiver bound, radio calibrated ☐ Mode switch → Manual ☐ `MOT_SLEWRATE` set

	#	Do	Expect	P / F	Values / notes
☐	5.1	Disarmed: dashboard thruster card (SERVO_OUTPUT_RAW AUX1/AUX2)	1500 both; VCB arms and reaches RUN with `cmd` 0/0	☐ P ☐ F	
☐	5.2	Arm in Manual; throttle forward a little	Both `cmd` positive, both motors forward	☐ P ☐ F	
☐	5.3	Throttle back	Both negative, through the 200 ms zero	☐ P ☐ F	
☐	5.4	Steering right, throttle centred	The pair that would swing the bow right (right negative / left positive). Mirrored: fix on the Rover side, not the VCB	☐ P ☐ F	
☐	5.5	Full stick forward, 10 s	SERVO_OUTPUT_RAW 1620, `cmd` 25, nothing above	☐ P ☐ F	
☐	5.6	Stick slam forward then back	`cmd` ramps at the slew rate; reversal through zero; no dropout	☐ P ☐ F	
☐	5.7	Transmitter off while armed and moving	Rover RC failsafe → outputs 1500 → `cmd` 0 (neutral, not garbage)	☐ P ☐ F	FS_ACTION:
☐	5.8	Disarm, arm again	`cmd` 0 while disarmed; commands resume on arm; no VCB action needed	☐ P ☐ F	
☐	5.9	30 min of gentle stick use	`bad` 0, `fault_code` 0, coils and precharge cool	☐ P ☐ F	

☐ **Gate 5 passed**: correct response to the sticks in all four directions at the 20 % limit, RC failsafe ends at neutral, 30 min clean.

Date _____ Signed _____ Witness _____

Stage 6 · Props on, in the water, 20 % limit

Boat tied at the dock, lines fore and aft, then open water. Same 20 % limit.

Date _____ Pilot _____ E-stop person _____ Location _____

	#	Do	Expect	P / F	Values / notes
☐	6.1	Props on; repeat 5.2–5.4 tied up	Lines go taut the right way; bow swings the right way	☐ P ☐ F	
☐	6.2	Dock drive, 30 min	`bus_mv` well above 35 V under load; `bad` 0; pack current noted	☐ P ☐ F	min bus V: max A:
☐	6.3	Open water in Manual; Rover tuning (CRUISE_SPEED, CRUISE_THROTTLE, steering, speed); Guided over MAVLink; MOOS via iArduRoverBridge	Tuned at 20 %; CRUISE_THROTTLE near the top of the band is expected	☐ P ☐ F	
☐	6.4	Autonomy acceptance run	Holds a line and a speed under Rover/MOOS control for a full run, sticks untouched	☐ P ☐ F	

☐ **Gate 6 passed**: control system tested and tuned under autonomy at 20 %. **Only now may the limit be raised.**

Date _____ Signed _____ Witness _____

Stage 7 · Raising the limit (after gate 6, not before)

`THRUST_LIMIT_PERMILLE` in `config.h`, rebuild, re-flash, then the matching SERVO MIN/MAX (`MAX = 1525 + limit × 0.475`, `MIN = 1475 − limit × 0.475`). At each step: 4.3, 4.4, 4.6 again with props off, then a dock drive, then retune.

	Limit	SERVO MIN / MAX	4.3 limit hold	4.4 end points (R+ R– L+ L–)	4.6 reversal	Dock drive	Retuned	Date / notes
☐	400	1285 / 1715	☐ P ☐ F	☐☐☐☐	☐ P ☐ F	☐ P ☐ F	☐	
☐	700	1143 / 1857	☐ P ☐ F	☐☐☐☐	☐ P ☐ F	☐ P ☐ F	☐	
☐	1000	1000 / 2000	☐ P ☐ F	☐☐☐☐	☐ P ☐ F	☐ P ☐ F	☐	

Open questions this plan answers (fill in as they are settled)

Question	Step	Answer
Which build is on the VCB today	0.1	
Console port and baud; does console input work	0.2, 0.3	
HSI timebase offset; deadband adequate	0.7	
E-stop / TSMS sense polarity; do the coils really lose power	1.1, 1.2, 1.4, 1.5	
What loses power on E-stop; what actually stops a turning motor	1.7, 3.8	
`bus_mv` calibration	2.2	
Can the precharge path take repeated attempts	2.10	
Physical left/right and forward direction	3.1–3.3	
Converter's reaction to a direction change; is the 200 ms hold needed	3.10, 4.6, 4.7	
Converter full scale 127 or 100	4.8	
Is "VCB stays in RUN when Rover disarms" acceptable	4.9	

Appendix · Command cheat sheet (test order)

Companion to `VCB_COMMISSIONING.md` (the plan) and `VCB_COMMISSIONING_CHECKLIST.pdf` (the printable). Step numbers match the plan. **M** = on the Mac in `~/moos/2.733`; **J** = on the Jetson, in the repo checkout there (`~/project-remora`; adjust if it lives elsewhere). Props off through stage 5. Nothing here raises the 20 % limit before stage 7.

Before every session (J)

```
ssh boat@highfieldboat.tailcacf9.ts.net           # or: mosh ... -- tmux new -A -s boat
cd ~/project-remora
deploy/update.sh                                  # fast-forward to origin/main, re-install, restart services
systemctl status vcb-logger --no-pager | head -3  # must be active; it writes ~/vcb.json
python3 -c 'import serial, pymavlink'             # bench tools need both (pip3 install --user pyserial
pymavlink)
```

Open three windows and keep them open all day:

```
# 1. raw VCB console: every line the VCB prints (ARMED, FAULT n, THRUSTER ..., BENCH ...)
tail -f "$(ls -t ~/vcblogs/*.csv | head -1)"

# 2. parsed fields: status (thruster_state, armed, lsc/hsc_closed, bus_mv, fault_code, cmd_source)
#    and pwm (chN_us, chN_valid, chN_raw_permille, chN_applied_permille, chN_bad, chN_cmd)
watch -n0.5 cat ~/vcb.json

# 3. a shell for sending console commands (the logger keeps reading; we only write)
stty -F /dev/vcb 115200 raw -echo clocal
vcb() { printf '%s\n' "$*" > /dev/vcb; }
vcb help
```

Dashboard: `http://highfieldboat.tailcacf9.ts.net:8080/` (thruster card = Pixhawk outputs + VCB).

Reset the VCB without touching power (needed after every latched fault 8, or `vcb clear` for fault 9):

```
openocd -f interface/stlink.cfg -f target/stm32h7x.cfg -c "init; reset run; exit"
# or: STM32_Programmer_CLI -c port=SWD -rst
```

Stage 0: firmware and dead-bus checks

```
# 0.1  dump what is on the board today (J) without resetting it, compare with the August dump
#      (mode=hotplug / halt+resume: the VCB keeps running; the August dump is in the checkout)
tail -3 "$(ls -t ~/vcblogs/*.csv | head -1)"      # first: what the running build says it is (old build:
mapping,...clamp_above_us=1200)
sudo openocd -f interface/stlink.cfg -f target/stm32h7x.cfg \
  -c "init; halt; dump_image $HOME/VCB_flash_$(date +%Y%m%d_%H%M%S).bin 0x08000000 0x100000; resume; exit"
sudo chown boat:boat ~/VCB_flash_*.bin
#   or: STM32_Programmer_CLI -c port=SWD mode=hotplug -u 0x08000000 0x100000 ~/VCB_flash_$(date
+%Y%m%d_%H%M%S).bin
cmp -l ~/VCB_flash_<new>.bin firmware/VCB_flash_20260805_211528_pre_tolerant_pwm.bin | wc -l   # 0 = same
build as August
cp ~/VCB_flash_<new>.bin firmware/ && git add firmware/VCB_flash_<new>.bin                    # keep it; commit
with the results

# 0.2  flash the committed binary (J); it is in the checkout, no copying needed
sha256sum firmware/VCB-pixhawk-0.3.1.bin          #
5cde2afaa82b94a7a8ddbb0515eed363c2a4f1c200fdaea2b164953e0a0f82b5
sudo openocd -f interface/stlink.cfg -f target/stm32h7x.cfg -c "program firmware/VCB-pixhawk-0.3.1.bin
0x08000000 verify reset exit"
#   or: STM32_Programmer_CLI -c port=SWD -w firmware/VCB-pixhawk-0.3.1.bin 0x08000000 -v -rst
#   (sudo until the STLINK udev rule below is in; the VCP /dev/vcb already has one)
#   window 1 must show:  VCB pixhawk-pwm reversible 0.3.1; ...
#   once only, so openocd works as boat from then on:
#   echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="0483", ATTR{idProduct}=="3754", MODE="0666"' | sudo tee
/etc/udev/rules.d/60-stlink.rules
#   sudo udevadm control --reload && sudo udevadm trigger
#   to go back:          STM32_Programmer_CLI -c port=SWD -w ~/VCB_flash_<new>.bin 0x08000000 -v -rst
#   to rebuild after a config change (M): cd ~/moos/2.733/firmware/VCB-pixhawk && make TOOLCHAIN=$TC/bin/
#     (TC = Arm GNU Toolchain 14.2 darwin-arm64 dir), cp build/VCB-pixhawk.bin ../VCB-pixhawk-<ver>.bin,
commit, push, update.sh
#     or on the Jetson: sudo apt install gcc-arm-none-eabi && make   (then flash build/VCB-pixhawk.bin)
```

```
# 0.3  console input works
vcb help                      # window 1: CONSOLE: bench on | bench off | bench R L ... | stop | clear | help

# 0.4  Pixhawk params for this build (only AFTER 0.2)
python3 tools/ap_apply.py params/vcb-pixhawk.parm --dry-run
python3 tools/ap_apply.py params/vcb-pixhawk.parm

# 0.5  safety switch off, otherwise there is no PWM at all (reboot the Pixhawk after)
printf 'BRD_SAFETY_DEFLT,0\n' > /tmp/safety.parm && python3 tools/ap_apply.py /tmp/safety.parm

# 0.6  dead-bus contract (pack disconnected; the VCB will sit in FAULT 3/6 latched, that is fine)
python3 tools/vcb_bench_check.py --pwm 1500,1550,1600,1620,1700,1450,1400,1380,1300,1000,2000
#   expect raw 0,52,157,200,368,-52,-157,-200,-368,-1000,1000; applied clamps at +-200; LEFT->ch2, RIGHT->ch1
python3 tools/vcb_bench_check.py --pwm 990,1000,1010,1990,2000,2010
#   expect raw -1000,-1000,-978,978,1000,1000; all valid=1; bad not counting
python3 tools/vcb_bench_check.py --pwm 2250,2300,2350,650 --hold 3      # only if ArduPilot accepts them
#   expect 2250/2300 valid raw 1000; 2350/650 bad counts, valid 0 after 100 ms

# 0.7  timebase: in the 1500 step above, ch1_us/ch2_us must read 1485..1515
```

Stage 1: safety, contactors open (Pixhawk unplugged from CONT)

```
# 1.1-1.3  physical: E-stop x3, TSMS x3, battery disconnect. Watch window 2:
#          status.estop_24v_present / tsms_24v_present / bus_mv ; meter on the coil feed
# 1.4-1.5  E-stop pressed (then TSMS pulled), plug the Pixhawk in at neutral:
#          window 1: ARMED -> THRUSTER STARTUP_LSC ... -> FAULT 3 ... x3 -> FAULT latched. Then unplug, reset:
openocd -f interface/stlink.cfg -f target/stm32h7x.cfg -c "init; reset run; exit"
# 1.6  software stop / clear
vcb stop                      # FAULT 9 ..., FAULT latched: stop command
vcb clear                     # CLEAR: stop un-latched ...
# 1.7  by inspection: what loses power on E-stop (converter? motors? coils only?) -> checklist
```

Stage 2: VCB arm, start-up, disarm, zero throttle (pack on)

Power-up order: Pixhawk (disarmed, outputs 1500) → TSMS in, E-stop released → VCB power.

```
# 2.1  hands off; window 1: ARMED ... STARTUP_LSC ... STARTUP_WAIT ... STARTUP_HSC ... RUN ; window 2: state
5, cmd 0/0
# 2.2  meter at the contactor output vs status.bus_mv (within 1 V)
# 2.3  10 min in RUN, fault_code stays 0
# 2.4  E-stop in RUN -> FAULT 8, lsc/hsc_closed 0, "FAULT latched". Release: must stay open. Reset:
openocd -f interface/stlink.cfg -f target/stm32h7x.cfg -c "init; reset run; exit"
# 2.5  same with TSMS
# 2.6  pull CONT in RUN -> FAULT 1 within 100 ms ; plug back -> "FAULT cleared" (3 s) -> ARMED (1 s) -> RUN
# 2.7  reboot the Pixhawk in RUN (QGC: Reboot Vehicle) -> FAULT 1 -> recovery
# 2.8  arm the Pixhawk in Manual and nudge throttle during the 1.5 s start-up -> "DISARMED: throttle left
neutral before RUN"
# 2.9  software stop with the bus live, then recovery
vcb stop                      # FAULT 9, contactors open
vcb clear                     # then 3 s neutral -> FAULT cleared -> ARMED -> RUN
# 2.10 pack disconnected, VCB reset -> FAULT 6 x3 -> FAULT latched; precharge resistor cool
# 2.11 bench-mode arm/disarm with nothing turning (Pixhawk unplugged from CONT)
python3 tools/vcb_bench_drive.py --right 0 --left 0 --seconds 5
#      window 1: BENCH: on ... ARMED ... RUN ... DISARMED: bench off ... back to IDLE ... BENCH: off
python3 tools/vcb_bench_drive.py --right 0 --left 0 --seconds 30     # then Ctrl-C mid-way: same clean exit
# 2.12 three power cycles, RUN each time
```

Stage 3: Jetson drives the motors at 5 %, props off

Pixhawk unplugged from CONT or left in: bench mode ignores the PWM inputs. 50 permille = 5 % = byte ±6.

```
# 3.1  right forward: which motor, which way?
python3 tools/vcb_bench_drive.py --right 50 --left 0 --seconds 5
# 3.2  left forward
python3 tools/vcb_bench_drive.py --right 0 --left 50 --seconds 5
# 3.3  reverse, each
python3 tools/vcb_bench_drive.py --right -50 --left 0 --seconds 5
python3 tools/vcb_bench_drive.py --right 0 --left -50 --seconds 5
# 3.4  both
python3 tools/vcb_bench_drive.py --right 50 --left 50 --seconds 10
```

```
# 3.5  30 s steady
python3 tools/vcb_bench_drive.py --right 50 --left 0 --seconds 30
# 3.6  the caps: 10 % runs; 30 % is refused by the tool and clamped by the firmware
python3 tools/vcb_bench_drive.py --right 100 --left 100 --seconds 5           # cmd 13/13
python3 tools/vcb_bench_drive.py --right 300 --left 0 --seconds 5             # "refusing: |permille| above --
max 100"
vcb bench on; vcb bench 300 0; vcb bench off                                  # window 1: BENCH: clamped to
100 0 (limit +-100)
# 3.7  dead Jetson process: start a 30 s run, then from another shell
python3 tools/vcb_bench_drive.py --right 50 --left 0 --seconds 30
kill -9 $(pgrep -f vcb_bench_drive.py)       # <= 500 ms later: FAULT 1 "bench command lost", motor stops
vcb bench off                                # tidy up; the VCB recovers to IDLE on its own after 3 s of
neutral
# 3.8  E-stop during a run -> FAULT 8, motor stops. Note what actually stopped it. Reset the VCB afterwards.
# 3.9  software stop during a run (second shell)
vcb stop                                     # FAULT 9, motor stops
vcb clear
# 3.10 direct reversal through the 200 ms zero hold (cmd 6 -> 0 -> -6), each side
python3 tools/vcb_bench_drive.py --right 50 --left 0 --seconds 3 --flip
python3 tools/vcb_bench_drive.py --right 0 --left 50 --seconds 3 --flip
```

If 3.1 shows the wrong motor or the wrong direction: it is fixed in `firmware/VCB-pixhawk/Core/Src/eprop.cpp` (addresses or the direction byte), rebuilt and re-flashed with the stage 0 commands, then 3.1–3.3 again.

Stage 4: Pixhawk motor test, props off (Pixhawk on CONT, `vcb bench off`)

`--live` requires the VCB in RUN with no fault and asks for a typed confirmation; it prints measured μs, raw/applied permille and `cmd` for each step.

```
# 4.1  right forward (AUX1 -> ch1 -> right motor?)
python3 tools/vcb_bench_check.py --live --side right --pwm 1550 --hold 2        # cmd 7
# 4.2  left forward, then each in reverse
python3 tools/vcb_bench_check.py --live --side left  --pwm 1550 --hold 2
python3 tools/vcb_bench_check.py --live --side right --pwm 1450 --hold 2        # cmd -7
python3 tools/vcb_bench_check.py --live --side left  --pwm 1450 --hold 2
# 4.3  the limit, 10 s
python3 tools/vcb_bench_check.py --live --side right --pwm 1620 --hold 10       # cmd 25
# 4.4  END POINTS, 30 s each, both sides: raw +-1000, applied +-200, cmd +-25, bad not counting, motor keeps
running
python3 tools/vcb_bench_check.py --live --side right --pwm 2000 --hold 30
python3 tools/vcb_bench_check.py --live --side right --pwm 1000 --hold 30
python3 tools/vcb_bench_check.py --live --side left  --pwm 2000 --hold 30
python3 tools/vcb_bench_check.py --live --side left  --pwm 1000 --hold 30
# 4.5  acceptance window (servo tester on CONT if ArduPilot refuses): 2200,2300,800,700 valid; 2350/650 ->
bad, FAULT 1
python3 tools/vcb_bench_check.py --live --side right --pwm 2200,2300,800,700 --hold 3
# 4.6  direct reversal at 20 %: cmd 25 -> 0 (200 ms) -> -25, no dropout, both sides
python3 tools/vcb_bench_check.py --live --side right --pwm 1620,1380 --hold 3
python3 tools/vcb_bench_check.py --live --side left  --pwm 1620,1380 --hold 3
# 4.7  (optional) is the hold needed? build without it, re-flash (stage 0.2), repeat 4.6, then put it back
sed -i '' 's/^#define DIRECTION_CHANGE_STOP_MS .*/#define DIRECTION_CHANGE_STOP_MS     0/' Core/Inc/config.h
# (M)
sed -i '' 's/^#define DIRECTION_CHANGE_STOP_MS .*/#define DIRECTION_CHANGE_STOP_MS     200/' Core/Inc/config.h
# (M) revert
#      (watch window 2: chN_glitch counts the discarded glitch pairs; chN_bad must stay 0 on a clean signal)
# 4.8  scale: at 1620 us the converter sees 25. If the motor behaves like 25 %, full scale is 100:
sed -i '' 's/^#define EPROP_FULL_SCALE_VALUE .*/#define EPROP_FULL_SCALE_VALUE       100/' Core/Inc/config.h
# (M), rebuild, re-flash
# 4.9  QGC: disarm while the VCB is in RUN -> outputs 1500, VCB stays RUN with cmd 0. Decide and write it
down.
```

Stage 5: RC, props off (receiver installed)

```
python3 tools/ap_inventory.py | grep -i -E "rc|receiver"            # receiver bound?
#   QGC: Radio calibration; Flight modes: Manual on the mode switch; arm switch if used
printf 'MOT_SLEWRATE,100\n' > /tmp/slew.parm && python3 tools/ap_apply.py /tmp/slew.parm
# 5.1-5.9  sticks; watch the dashboard thruster card (SERVO_OUTPUT_RAW 1380..1620) and window 2 (cmd)
#   5.7: transmitter off while armed -> FS action -> outputs 1500 -> cmd 0
python3 tools/ap_logs.py --latest 1                                  # keep the log of the session
```

Stage 6: props on, water, 20 % (nothing new to type; Rover tuning in QGC, logs with ap_logs.py)

Stage 7: raising the limit (after gate 6 only)

One config line, rebuild, re-flash (stage 0.2), then the matching Pixhawk range:

```
# 400 permille
sed -i '' 's/^#define THRUST_LIMIT_PERMILLE .*/#define THRUST_LIMIT_PERMILLE        400/' Core/Inc/config.h
# (M)
printf 'SERVO9_MIN,1285\nSERVO9_MAX,1715\nSERVO10_MIN,1285\nSERVO10_MAX,1715\n' > /tmp/limit.parm && python3
tools/ap_apply.py /tmp/limit.parm   # (J)
# 700 permille
sed -i '' 's/^#define THRUST_LIMIT_PERMILLE .*/#define THRUST_LIMIT_PERMILLE        700/' Core/Inc/config.h
printf 'SERVO9_MIN,1143\nSERVO9_MAX,1857\nSERVO10_MIN,1143\nSERVO10_MAX,1857\n' > /tmp/limit.parm && python3
tools/ap_apply.py /tmp/limit.parm
# 1000 permille
sed -i '' 's/^#define THRUST_LIMIT_PERMILLE .*/#define THRUST_LIMIT_PERMILLE        1000/' Core/Inc/config.h
printf 'SERVO9_MIN,1000\nSERVO9_MAX,2000\nSERVO10_MIN,1000\nSERVO10_MAX,2000\n' > /tmp/limit.parm && python3
tools/ap_apply.py /tmp/limit.parm
# at each step: 4.3, 4.4, 4.6 again (props off), then a dock drive, then retune
```

Fault codes (window 1 prints `FAULT n: reason`)

n	meaning	clears by
1	command lost (PWM invalid, or no `bench R L` for 500 ms)	valid neutral command for 3 s
2–7	contactor / bus interlock during start-up or RUN	3 s neutral, max 3 retries per boot, then reset
8	contactor opened in RUN (E-stop / TSMS)	reset only
9	`stop` on the console	`vcb clear`, then 3 s neutral